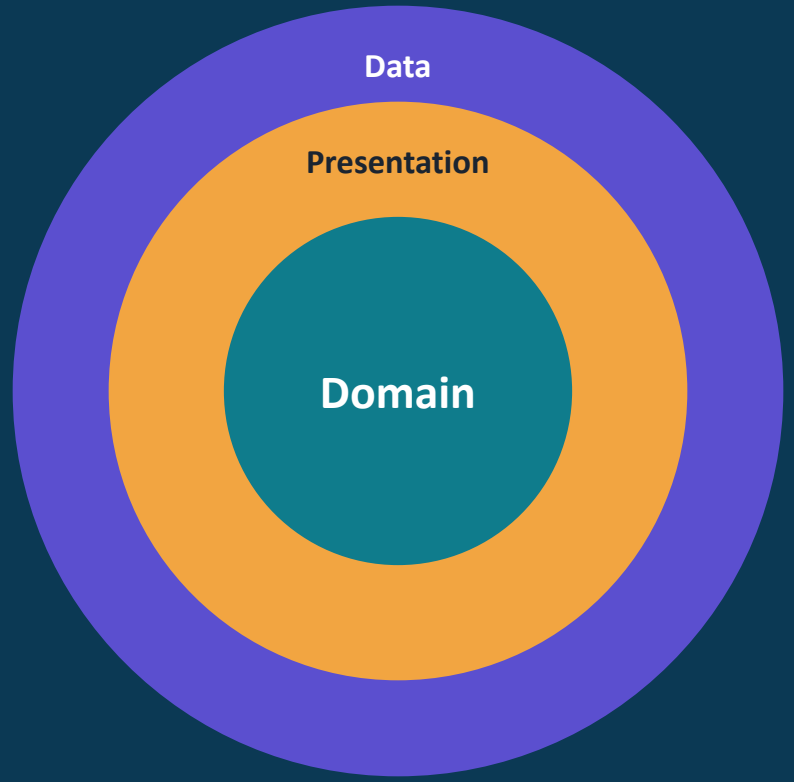
Clean Architecture di Flutter

Pemrograman Mobile ·

Memisahkan kode agar rapi, mudah diuji, dan mudah dikembangkan

Oleh : Luqman Affandi, S.Kom., MMSI



Dependensi selalu mengarah ke dalam

Tujuan Pembelajaran



01

Menjelaskan prinsip SOLID dan separation of concerns dengan contoh Flutter



02

Membedakan struktur feature-first dan layer-first beserta untung-ruginya



03

Menjelaskan 3 layer (presentation, domain, data) dan aturan dependensi



04

Membedakan entity dan model, serta peran repository, use case, dan DI



05

Merefactor project Minggu 5/6 menjadi feature-first yang terpisah layer-nya



06

Menguji use case dengan repository palsu, tanpa database sungguhan

Masalah: Satu Widget Mengerjakan Semuanya

notes_page.dart (sebelum refactor)

```
class NotesPage extends StatelessWidget {
  Widget build(BuildContext context) {
    // 1. buka database SQLite
    final db = openDatabase('notes.db');
    // 2. query SQL
    // 3. parsing Map / JSON
    // 4. format tanggal
    // 5. baru menampilkan ListView
    // ... 400 baris ...
  }
}
```

Sulit diuji

Untuk mengetes tampilan harus menyiapkan database asli.

Sulit diubah

Ganti SQLite ke API berarti membongkar halaman UI.

Sulit dibaca

Satu file ratusan baris dengan banyak tanggung jawab.

Clean Architecture = membagi tugas. Setiap bagian punya satu tanggung jawab.

Analogi Mudah: Sebuah Restoran



Presentation

Pelayan

Menerima pesanan dan menyajikan makanan. Tidak memasak, tidak belanja.

Di Flutter: Widget, Notifier, Router



Domain

Kepala Dapur & Resep

Menentukan apa yang dibuat dan aturannya. Inti bisnis restoran.

Di Flutter: Entity, Use case, Kontrak repository



Data

Gudang & Pemasok

Tempat bahan diambil. Bisa pasar, bisa supermarket — boleh berganti.

Di Flutter: Model, Repository impl, SQLite / Dio

Ganti pemasok? Resep dan pelayan tetap. Ganti SQLite ke API? Domain dan UI tetap.

01

Konsep Dasar

SOLID, tiga layer, struktur folder, dan kosakata peran

SOLID: Lima Prinsip dalam Bahasa Sehari-hari

		Arti praktis	Gejala pelanggaran di Flutter
S	**Single Responsibility**	Satu kelas, satu alasan untuk berubah	Widget 400 baris: fetch API + parsing JSON + format tanggal
O	**Open/Closed**	Tambah perilaku lewat ekstensi, bukan mengedit kode lama	Sumber data baru memaksa edit widget yang sudah jalan
L	**Liskov Substitution**	Pengganti harus bisa menggantikan aslinya tanpa merusak	Repository palsu melempar error yang tak pernah dilempar aslinya
I	**Interface Segregation**	Jangan paksa klien bergantung pada yang tak dipakai	Satu AppRepository raksasa untuk auth + notes + push
D	**Dependency Inversion**	Bergantung pada abstraksi, bukan implementasi	Notifier memanggil Dio / sqflite langsung

Contoh Dependency Inversion

Sebelum

```
class NotesNotifier {
  // langsung membuat Dio sendiri
  final dio = Dio();

  Future<void> load() async {
    final res = await dio.get('/notes');
    // parsing di sini...
  }
}
```

Sesudah

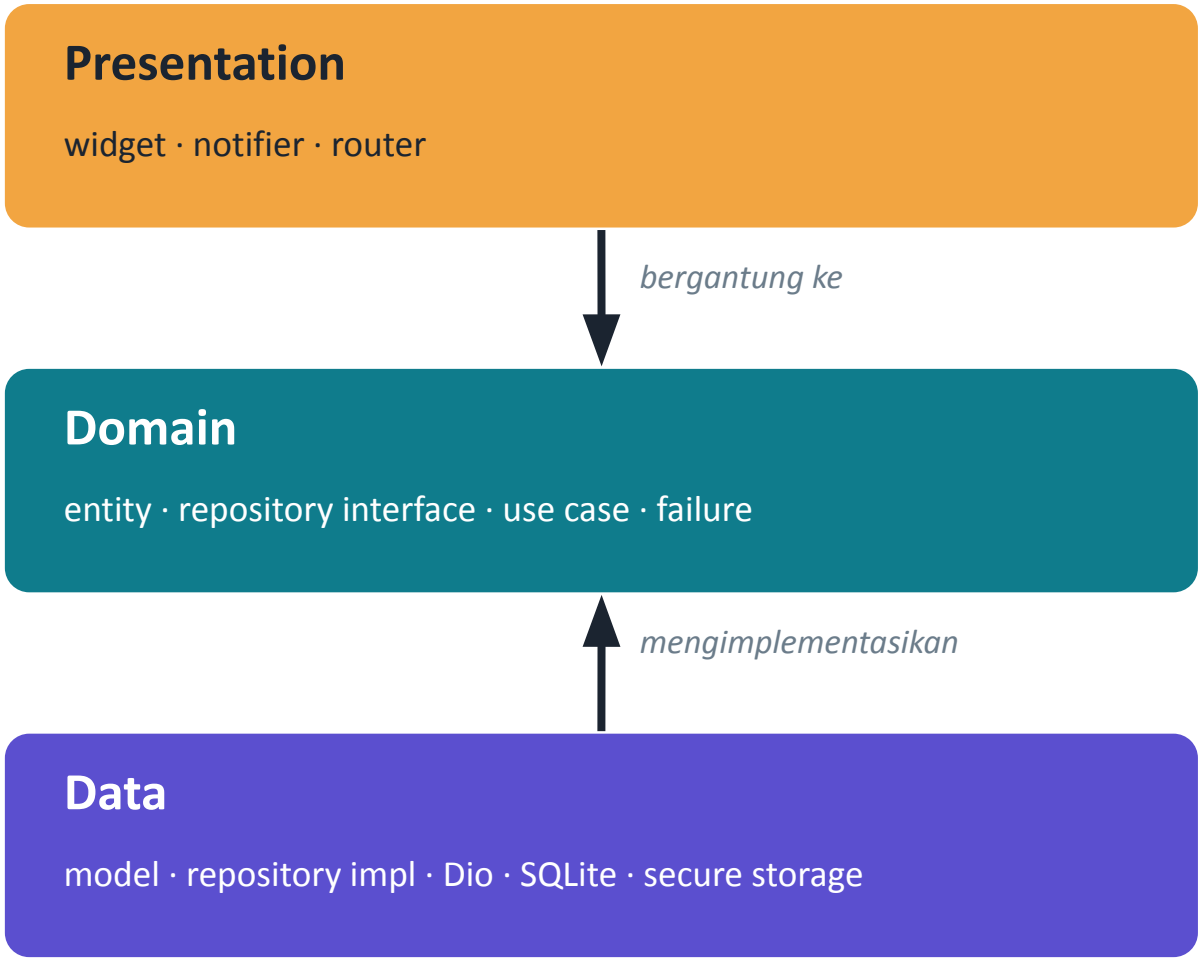
```
class NotesNotifier {
  // menerima abstraksi dari luar
  NotesNotifier(this.repo);
  final NoteRepository repo;

  Future<void> load() =>
      repo.fetchNotes();
}
```



Seperti colokan listrik: charger cukup tahu bentuk stopkontak, tidak perlu tahu listriknya dari PLN atau genset. Notifier cukup tahu NoteRepository — datanya dari SQLite, API, atau data palsu untuk test.

Tiga Layer dan Aturan Dependensi



Aturan emas: dependency rule

- Dependensi hanya mengarah ke dalam, yaitu ke domain.
- Domain tidak tahu apa pun tentang Flutter, Dio, atau SQLite.
- Karena murni Dart, logika bisnis bisa diuji tanpa emulator, jaringan, atau database.
- Data layer yang "menyesuaikan diri" dengan kontrak domain, bukan sebaliknya.

Siapa Boleh Mengenal Siapa?

Layer	Boleh mengenal	Tidak boleh mengenal
Presentation	Domain (entity, use case), Flutter, Riverpod	Dio, SQLite, JSON mentah (kecuali file wiring DI)
Domain	Dart murni saja	Flutter, Dio, sqflite, Firebase
Data	Domain (entity, kontrak), Dio, sqflite	Widget dan apa pun di presentation



Bukan sekadar pindah folder. Memindahkan file tanpa memutus dependensi terlarang (widget → Dio, notifier → SQLite) bukan Clean Architecture. Yang dinilai adalah arah panah dependensi, bukan kedalaman folder.

Feature-first vs Layer-first

Layer-first

```
lib/
├── data/
├── domain/
└── presentation/
```

- Cocok untuk project kecil satu fitur.
- Cepat dibuat, tetapi berantakan saat fitur bertambah.
- Seperti pindahan dengan satu kardus khusus piring, satu khusus baju — untuk satu kamar harus buka semua kardus.

Feature-first (dipakai di codelab)

```
lib/features/
├── notes/
│   ├── data/
│   ├── domain/
│   └── presentation/
└── auth/
    ├── data/ ...
```

- Cocok untuk project multi-fitur.
- Satu fitur bisa dipahami atau dihapus tanpa menyentuh fitur lain.
- Seperti satu kardus per kamar — mau pindah kamar, angkat satu kardus saja.

Kosakata Peran

Domain

Entity

Objek bisnis murni, tanpa toMap / fromMap / toJson.

Analogi: Isi pesanan: nama menu, jumlah

Data

Model

Bentuk data dari luar (JSON / SQLite) + konversi ke entity.

Analogi: Bon gudang dengan kode khusus

Domain

Repository interface

Kontrak: APA yang dibutuhkan domain, mis. fetchNotes().

Analogi: Daftar belanja

Data

Repository impl

BAGAIMANA kontrak dipenuhi memakai Dio / SQLite.

Analogi: Kurir yang berbelanja

Domain

Use case

Satu operasi bisnis, mis. GetNotes, SyncNotes.

Analogi: Satu resep masakan

Presentation

Dependency Injection

Dependensi disuntikkan lewat provider Riverpod. Widget tidak pernah new Repository().

Analogi: Manajer yang menugaskan kurir

Entity vs Model

Domain **Entity — note.dart**

```dart
class Note {
  final int? id;
  final String title;
  final DateTime updatedAt;
  // tanpa toMap / fromMap
}
```

toEntity()

Data **Model — note_model.dart**

```dart
class NoteModel extends Note {
  Map<String, Object?> toMap() => {..};
  factory NoteModel.fromMap(map) {..}
  Note toEntity() => Note(..);
}
```

Analogi: entity seperti KTP — data inti seseorang yang dipakai di mana-mana. Model seperti formulir pendaftaran di satu kantor tertentu: kolomnya mengikuti aturan kantor itu (di SQLite, tanggal disimpan sebagai teks dan boolean sebagai 0/1). Formulir boleh berganti, KTP tetap.

02

Praktikum 1

Petakan file, temukan pelanggaran, gambar struktur target

Langkah 1: Petakan File ke Layer

File	Layer saat ini	Masalah
pages/home_page.dart	presentation	Memanggil Dio langsung? Format tanggal + parsing JSON di widget?
data/api_client.dart	data	OK bila hanya dipakai repository, bukan widget
providers/auth_provider.dart	presentation (state)	OK bila hanya memanggil repository / use case
data/auth_repository.dart	data (+ kontrak tercampur)	Interface dan implementasi masih satu kelas



Tugas: isi tabel ini untuk project Anda sendiri dan simpan di README. Tabel ini menjadi daftar pekerjaan refactor.

Langkah 2: Tandai Tiga Pelanggaran Klasik

Widget menyentuh jaringan / database

```
rg
"Dio\(|http\.|openDatabase|SharedPreferences\.getInstance|FlutterSecu
reStorage" lib/pages lib/widgets
```

Logika bisnis di dalam build()

```
rg "DateFormat|jsonDecode|\.toIso8601String" lib/pages lib/widgets
```

Instansiasi manual (DI bocor)

```
rg "Repository\(|Dio\(BaseOptions" lib/pages lib/providers
```

Setiap temuan = satu item refactor. Target: nol hasil di folder presentation.

Langkah 3: Gambar Struktur Target

```
lib/
├── core/
│   └── failures.dart
├── features/
│   └── notes/
│       ├── domain/
│       │   ├── entities/note.dart
│       │   ├── repositories/note_repository.dart
│       │   └── usecases/get_notes.dart
│       ├── data/
│       │   ├── models/note_model.dart
│       │   └── repositories/note_repository_impl.dart
│       └── presentation/
│           ├── providers/notes_providers.dart
│           └── pages/notes_page.dart
└── routes.dart
```

Domain Aturan bisnis: entity, kontrak, use case

Data Cara mengambil data: model, impl SQLite / Dio

Presentation Tampilan dan state: provider, halaman



Jangan buat fitur baru

Praktikum ini memindahkan dan memisahkan kode yang sudah jalan. Commit dulu agar bisa dibandingkan dan di-rollback:

```
git commit -m "chore: snapshot
before clean architecture"
```

03

Praktikum 2

Membangun domain dan data layer: entity, failure, kontrak, model, implementasi, use case

Entity Murni (Domain)

lib/features/notes/domain/entities/note.dart

```
class Note {
  const Note({
    this.id,
    required this.title,
    this.body = '',
    required this.updatedAt,
    this.dirty = false,
  });

  final int? id;
  final String title;
  final String body;
  final DateTime updatedAt;
  final bool dirty;
}
```



Tanpa import Flutter

Bisa dipakai dan diuji di Dart murni.



Tanpa toMap / fromMap

Urusan format database bukan tugas entity.



Semua field final

Objek tidak berubah diam-diam (immutable).

Failure dan Kontrak Repository (Domain)

lib/core/failures.dart

```
sealed class Failure {
  const Failure(this.message);
  final String message;
}
class LocalFailure extends Failure {
  const LocalFailure(super.message);
}
class NetworkFailure extends Failure {
  const NetworkFailure(super.message);
}
```

.../domain/repositories/note_repository.dart

```
abstract class NoteRepository {
  Future<({List<Note> notes,
           Failure? failure})> fetchNotes();

  Future<({Note? note,
           Failure? failure})> addNote({
    required String title,
    String body = '',
  });
}
```

Record ({notes, failure}) = paket dengan dua kantong. Kalau berhasil, kantong notes berisi data dan failure kosong. Kalau gagal, failure berisi pesan. Alternatif di industri: Either<Failure, T> dari package fpdart / dartz.

Model: Tempat Satu-satunya untuk Mapping (Data)

lib/features/notes/data/models/note_model.dart (disederhanakan)

```
class NoteModel extends Note {
  // constructor: sama seperti Note
  Map<String, Object?> toMap() => {
    'id': id,
    'title': title,
    'body': body,
    'updated_at': updatedAt.toIso8601String(),
    'dirty': dirty ? 1 : 0,
  };

  factory NoteModel.fromMap(Map<String, Object?> map) {
    return NoteModel(
      id: (map['id'] as num?)?.toInt(),
      title: map['title'] as String? ?? '',
      updatedAt: DateTime.tryParse(
        map['updated_at'] as String? ?? '')
        ?? DateTime.fromMillisecondsSinceEpoch(0),
      dirty: map['dirty'] == 1,
    );
  }

  Note toEntity() => Note(id: id, title: title, ...);
}
```

`toMap()`

Note → baris tabel SQLite

`fromMap()`

baris tabel SQLite → NoteModel

`toEntity()`

NoteModel → Note bersih untuk domain

Tips: tanda ?? memberi nilai cadangan jika data di database kosong, sehingga aplikasi tidak crash.

Repository Impl: Memenuhi Kontrak (Data)

.../data/repositories/note_repository_impl.dart (fetchNotes)

```
class NoteRepositoryImpl implements NoteRepository {
  NoteRepositoryImpl({required this.openDb});
  final Future<Database> Function() openDb;

  @override
  Future<({List<Note> notes, Failure? failure})>
      fetchNotes() async {
    try {
      final db = await openDb();                    // 1
      final rows = await db.query('notes',
          orderBy: 'updated_at DESC');              // 2
      final notes = rows
          .map((r) => NoteModel.fromMap(r).toEntity())
          .toList();                                // 3
      return (notes: notes, failure: null);
    } catch (e) {                                   // 4
      return (notes: const <Note>[],
        failure: LocalFailure('Gagal membaca: $e'));
    }
  }
}
```

1. Buka database (fungsi disuntikkan dari luar)
2. Ambil baris dari tabel notes
3. Ubah Map → NoteModel → Note entity
4. Jika gagal: kembalikan LocalFailure, bukan melempar exception

Use Case: Satu Operasi Bisnis (Domain)

.../domain/usecases/get_notes.dart

```
class GetNotes {
  const GetNotes(this._repository);
  final NoteRepository _repository;

  Future<({List<Note> notes, Failure? failure})>
      call() {
    return _repository.fetchNotes();
  }
}
```

Method call() membuat objek bisa dipanggil seperti fungsi: getNotes() sama dengan getNotes.call().



Use case perlu bila…

- menggabungkan lebih dari satu repository
- ada aturan bisnis: validasi, sorting, sync



Belum perlu bila…

operasinya CRUD satu baris. Repository boleh langsung dipanggil notifier — tulis alasannya.

Analogi: untuk "ambilkan segelas air" tidak perlu buku resep.

04

Praktikum 3

Wiring dengan Riverpod, halaman yang bersih, dan verifikasi

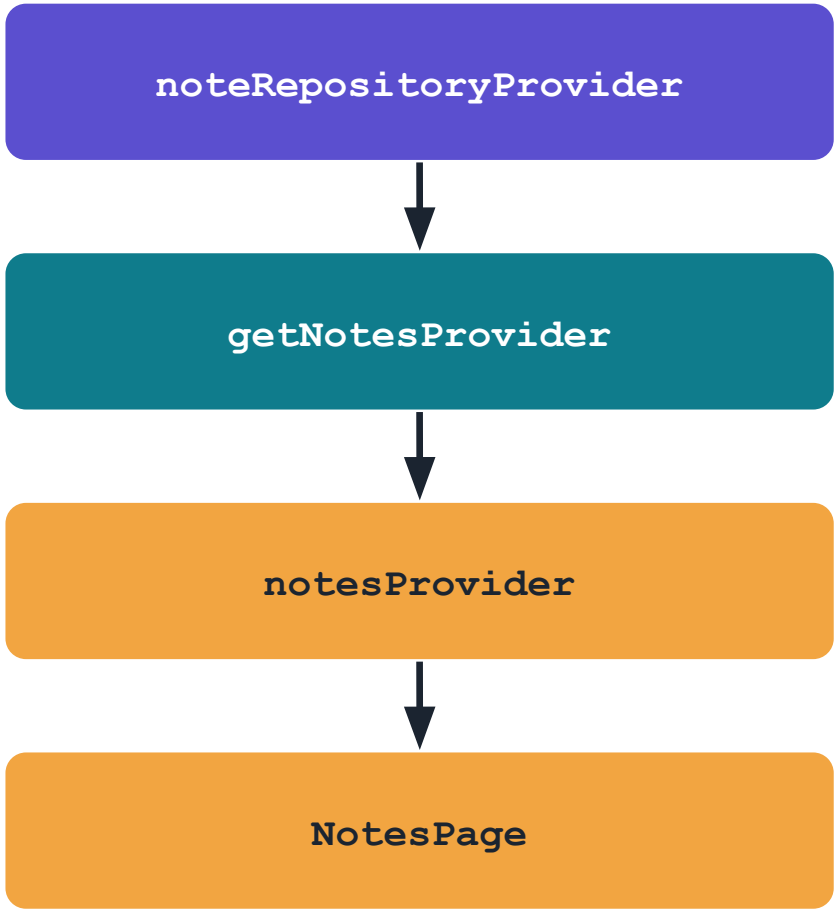
Dependency Injection dengan Riverpod

.../presentation/providers/notes_providers.dart

```dart
// Data: implementasi nyata
final noteRepositoryProvider =
    Provider<NoteRepository>((ref) {
  return NoteRepositoryImpl(openDb: openNotesDb);
});

// Domain: use case menerima abstraksi
final getNotesProvider = Provider<GetNotes>((ref) {
  return GetNotes(ref.watch(noteRepositoryProvider));
});

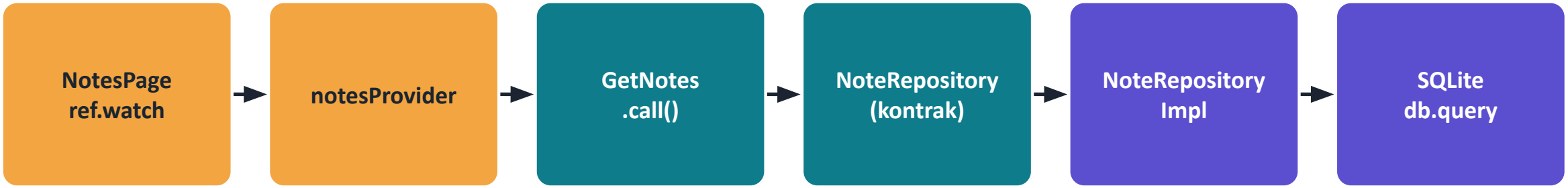
// Presentation: state untuk UI
final notesProvider =
    FutureProvider<List<Note>>((ref) async {
  final r = await ref.watch(getNotesProvider).call();
  if (r.failure != null) {
    throw Exception(r.failure!.message);
  }
  return r.notes;
});
```



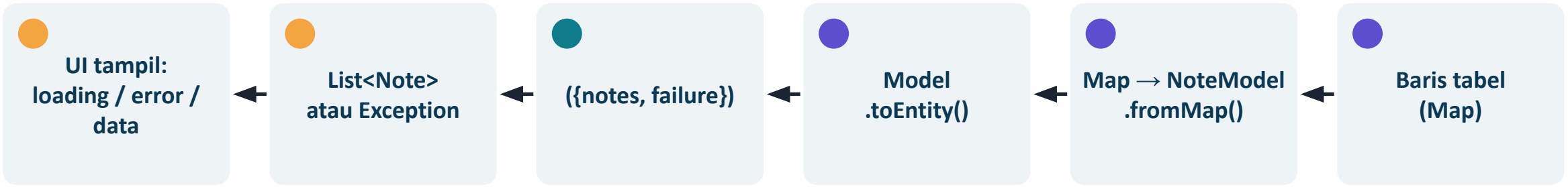
Widget tidak pernah membuat repository sendiri.

Alur Data Saat Halaman Dibuka

Permintaan



Hasil kembali



Presentation | Domain | Data

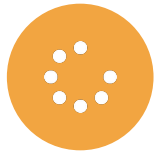
Domain di tengah tidak tahu apakah data berasal dari SQLite, API, atau data palsu.

Halaman Hanya Membaca State

.../presentation/pages/notes_page.dart

```
class NotesPage extends ConsumerWidget {
  const NotesPage({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final state = ref.watch(notesProvider);
    return state.when(
      loading: () => const Center(
          child: CircularProgressIndicator()),
      error: (e, _) => ErrorView(
        message: '$e',
        onRetry: () => ref.invalidate(notesProvider),
      ),
      data: (notes) => notes.isEmpty
          ? const Center(child: Text('Belum ada catatan.'))
          : NotesList(notes),
    );
  }
}
```



`loading`

Tampilkan indikator berputar



`error`

Tampilkan pesan + tombol coba lagi



`data`

Tampilkan daftar, atau pesan kosong

Tanpa SQL, Dio, JSON, atau format tanggal mentah.

Verifikasi Dependency Rule

1. Presentation steril

```
rg
"Dio\(|openDatabase|getDatabasesPath|FlutterSecureStorage|SharedPreferences\.getInsta
nce|jsonDecode" lib/features/*/presentation lib/pages
```

2. Domain steril

```
rg "import 'package:flutter|import 'package:dio|import 'package:sqflite|import
'package:firebase" lib/features/*/domain lib/core
```

3. Analisis + test

```
flutter analyze
flutter test
```



Definisi selesai: aplikasi berjalan sama seperti sebelum refactor (uji manual ±5 menit per fitur), dua grep nol hasil, flutter analyze bersih, test hijau, dan screenshot before/after di folder screenshots/.

05

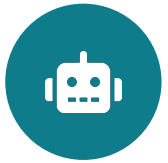
Peran AI

AI boleh mengusulkan, Anda yang memutuskan

AI Prompt Challenge

Prompt (diringkas)

```
Project Flutter saya: campus_notify (auth + FCM + pengumuman).
Kondisi kini: folder lib/{data, providers, pages, messaging},
repository tercampur dengan implementasi,
widget memanggil Dio langsung.
Tugas:
1. Usulkan struktur feature-first Clean Architecture
   (presentation/domain/data) untuk auth + announcements.
2. Untuk tiap file lama, sebutkan tujuan barunya.
3. Tandai bagian yang over-engineering untuk CRUD sederhana.
4. Tunjukkan wiring DI dengan Riverpod.
Jelaskan trade-off setiap keputusan.
```



AI mengusulkan, Anda memutuskan

Simpan di folder docs/:

- prompt dan output awal AI
- tabel usulan AI vs keputusan final
- hasil tiga grep verifikasi

Saat demo, Anda harus mampu menggambar arah panah dependensi tanpa membuka catatan.

Checklist Sebelum Menerima Usulan AI

- Interface repository di domain, implementasi di data (tolak bila satu folder)
- Domain bebas import Flutter / Dio / SQLite / Firebase — cek dengan grep
- Tidak ada use case untuk setiap CRUD satu baris (over-engineering)
- Entity bebas mapping: toMap / fromMap / toJson hanya di model
- Wiring DI terpusat di provider; widget tidak new Repository()
- Keputusan final dan alasan teknis Anda tercatat

Refactoring Challenge



01

Fitur kedua

Terapkan pola yang sama ke auth atau announcements: entity, interface, impl, 1 use case, wiring provider.



02

Fungsi murni

Pindahkan format tanggal dan parsing rute ke lib/core/format.dart agar bisa diuji tanpa widget.



03

Failure ramah

Ubah semua exception mentah menjadi Failure sebelum sampai UI. UI hanya menerima message.

Jangan biarkan satu fitur bersih sementara fitur lain masih kotor.

06

Penutup

Error umum, mini project, dan refleksi

Error Umum dan Solusinya

Gejala	Penyebab umum	Solusi
Import cycle / file saling mengimpor	Domain atau data mengimpor presentation	Cek arah panah: presentation → domain ← data
NoteModel can't be assigned to Note	Impl mengembalikan model, kontrak meminta entity	Selalu panggil .toEntity() di batas data → domain
ref.watch di luar widget / provider	Wiring DI ditulis di fungsi biasa	Wiring hanya di Provider / Notifier / ConsumerWidget
Test butuh database sungguhan	Use case diuji dengan impl asli	Uji domain dengan fake yang implements interface
Refactor merusak fitur	Tidak ada snapshot commit dan uji manual	Commit snapshot dulu, uji tiap fitur ±5 menit

Mini Project: Refactor Project Anda



1

1 fitur penuh

domain + data + presentation terpisah



2

Bukti dependency rule

hasil grep sterilitas di README



3

Perilaku identik

screenshot before/after per fitur



4

Minimal 2 test lulus

use case sukses + failure (repo palsu)



5

AI Challenge

prompt + usulan vs keputusan di docs/



6

Push ke portfolio

lib/, test/, docs/, README, screenshots/

Refleksi

- Mengapa interface repository harus tinggal di domain, bukan di data? Apa yang rusak bila dibalik?
- Kapan use case benar-benar dibutuhkan, dan kapan repository langsung ke notifier sudah cukup?
- Apa biaya over-engineering bagi tim kecil? Kapan biayanya sepadan?
- Bagian mana dari usulan AI yang Anda tolak atau sederhanakan, dan mengapa?

Ingat: yang dinilai adalah arah panah dependensi, bukan kedalaman folder.